

Modul Praktikum 2: Program Control, Selection and Repetition / Pointers and Array dalam Pemrograman C

- LO 2: Analyze programs to solve problems using the C language and AI problem-solving sessions related to the topics.

Sesi Kolaborasi AI: Debugging Logika dan Analisis Alamat Memori

Konsep Inti: Penggunaan AI dalam modul ini berfokus pada **LO 2 (Menganalisis program untuk memecahkan masalah)**. AI akan digunakan untuk melacak aliran program (*flow tracing*) dan mendiagnosis kesalahan yang sulit dilihat secara manual, seperti *infinite loop* atau *pointer* yang salah dereferensi.

Komponen HAICOLA	Fokus Pembelajaran	Aktivitas Kolaborasi AI & Refleksi
H-Holistic Abstraction	Kontrol Logika: Pahami bagaimana if-else bekerja secara eksklusif dan bagaimana switch-case memerlukan break untuk mencegah <i>fall-through</i> .	Tugas AI: Minta AI untuk menjelaskan skenario di mana menghilangkan break dari switch-case dapat menyebabkan kesalahan logika. Refleksi: Buat catatan tentang pentingnya break dalam logika switch .
A-AI-Assisted Debugging (Repetisi)	Dianosa Infinite Loop: Fokus pada kesalahan inisialisasi, kondisi, atau peningkatan/penurunan dalam <i>loop</i> (for , while , do-while).	Tugas AI: Ambil Contoh Program 1 (while (1)) dan buat <i>error</i> dengan menghilangkan attempt++ di blok else . Permintaan: "Kode saya terjebak dalam perulangan tak terbatas setelah 3 kali salah. Di mana <i>loop control variable</i> saya salah kelola?"
I-Introspection & Correction (Pointer)	Analisis Alamat Memori: Menguasai perbedaan antara nilai (*ptr) dan alamat (ptr) serta cara kerja Pass-by-Reference (Contoh Program swap).	Tugas AI: Berikan kode fungsi swap yang bermasalah (misalnya, lupa menggunakan * saat <i>swapping</i>). Permintaan: "Fungsi swap saya tidak berhasil menukar nilai x dan y di main . Jelaskan mengapa dan bagaimana pointer harus

		didereferensi (menggunakan *) untuk memanipulasi nilai asli."
C-Concept Expansion (Pointers & Array)	Analisis <i>Pointer Arithmetic</i>: Memahami bahwa nama <i>array</i> adalah <i>pointer</i> konstan ke elemen pertamanya, dan $*(ptr + i)$ identik dengan <code>arr[i]</code> .	Tugas AI: Ambil Contoh Program Pointer dalam Array . Permintaan: "Saya ingin mengakses elemen ke-5 dari array menggunakan <code>ptr + 4</code> . Berapa banyak <i>byte</i> yang <i>secara fisik</i> ditambahkan oleh C ke alamat <code>ptr</code> untuk mendapatkan alamat elemen ke-5? Jelaskan konsep <i>pointer arithmetic</i> ."
O-Operator Evaluation (Selection)	Pemilihan Struktur Kontrol Efisien: Mengevaluasi kapan harus menggunakan <code>switch-case</code> daripada <code>if-else if</code> .	Tugas AI: Berikan skenario (misalnya, penentuan hari dari 1-7). Permintaan: "Untuk kasus ini, mana yang lebih efisien dan <i>cleaner</i> secara kode, menggunakan <code>switch-case</code> atau deretan <code>if-else if</code> ?"
LA-Learned Application	Verifikasi Batas <i>Array</i>: Menerapkan kontrol perulangan yang benar untuk <i>array</i> untuk menghindari Off-by-One Errors (OBOE) .	Final Check AI: Minta AI untuk membuat kode <i>loop</i> for yang benar untuk mengiterasi <i>array</i> berukuran <i>N</i> . Permintaan: "Buatkan saya kode untuk mencetak semua elemen array berukuran 10. Jelaskan mengapa $i < 10$ lebih aman daripada $i \leq 10$ untuk mencegah akses di luar batas (LO 2)."

Poin Penting untuk Kolaborasi AI yang Efektif

1. **Lacak *Flow Control*:** Ketika *debugging loop* atau *selection*, minta AI untuk melakukan *step-by-step tracing* nilai variabel Anda. Ini sangat membantu untuk mendeteksi *off-by-one errors* (OBOE).
2. **Visualisasikan Memori:** Ketika bekerja dengan *pointer*, gunakan AI untuk memvisualisasikan **alamat** (misalnya, `0x7ffee...`) dan **nilai** yang disimpan di alamat tersebut. Ini menguatkan pemahaman Anda tentang **dereferensi (*)** dan **pengambilan alamat (&)**.

3. **Analisis Kompleksitas:** Gunakan AI untuk membandingkan tiga jenis *loop* (for, while, do-while) dan kapan masing-masing paling cocok digunakan. Ini adalah bagian dari **LO 3: Evaluasi jenis algoritma**.

Deskripsi:

Menguasai penggunaan kontrol program melalui struktur seleksi dan repetisi, memahami penggunaan pointer dalam manipulasi array, dan mengembangkan keterampilan dalam mendiagnosis serta memperbaiki kesalahan dalam program C.

Tujuan Pembelajaran:

- Memperdalam pemahaman tentang struktur kontrol program dalam bahasa C, meliputi seleksi dan repetisi.
- Menjelaskan secara rinci penggunaan pointers dalam manipulasi array dan aplikasi praktisnya.
- Melatih mahasiswa dalam menulis algoritma yang efisien menggunakan struktur data dan kontrol program yang kompleks.

Bagian 1: Kontrol Program – Seleksi

Sub-Materi: Pengenalan *if*, *else*, dan *else if*

- *if*: Blok kode yang hanya dieksekusi jika kondisi tertentu terpenuhi (true).
- *else*: Alternatif blok kode yang dieksekusi jika kondisi if tidak terpenuhi.
- *else if*: Menyediakan kondisi tambahan yang akan diperiksa jika kondisi if sebelumnya gagal.

Contoh Program:

```
#include <stdio.h>

int main() {
    int number;
    printf("Enter an integer: ");
    scanf("%d", &number);

    if (number % 2 == 0) {
        printf("The number is even.\n");
    } else {
        printf("The number is odd.\n");
    }

    return 0;
}
```

- Program ini meminta pengguna untuk memasukkan sebuah bilangan bulat.
- Menggunakan operator modulus % untuk mengecek apakah bilangan tersebut genap atau ganjil.
- if digunakan untuk mengevaluasi kondisi dan else untuk alternatif jika kondisi tidak terpenuhi.

Sub-Materi: Penggunaan *switch-case*

Tujuan penggunaan switch-case yaitu sebagai alternatif kepada struktur if yang lebih detail ketika membandingkan satu variabel dengan beberapa konstanta (lebih dari satu).

Contoh Program:

```
char grade;
printf("Enter your grade (A/B/C/D/F): ");
scanf(" %c", &grade);
switch(grade) {
    case 'A':
        printf("Excellent!\n");
        break;
    case 'B':
    case 'C':
        printf("Well done\n");
        break;
    case 'D':
        printf("You passed\n");
        break;
    case 'F':
        printf("Better try again\n");
        break;
    default:
        printf("Invalid grade\n");
}
```

Program menilai grade dan mencetak pesan yang relevan untuk setiap kondisi. switch-case efisien untuk pengujian nilai lebih dari satu variabel terhadap beberapa konstanta. Biasanya dapat digunakan pada studi kasus pemberian nilai berdasarkan nilai.

Bagian 2: Kontrol Program – Repetisi

Sub-Materi: Loop dengan for

Program ini menggunakan loop for untuk menghitung jumlah bilangan ganjil antara 1 dan 100. Tujuan dari contoh ini adalah untuk mengilustrasikan bagaimana loop for digunakan dalam kasus di mana kita tahu berapa kali loop harus diulang.

Contoh Program:

```
#include <stdio.h>

int main() {
    int count = 0; // Variabel untuk menyimpan jumlah bilangan ganjil

    // Loop dari 1 sampai 100
    for (int i = 1; i <= 100; i++) {
        if (i % 2 != 0) { // Jika i adalah bilangan ganjil
            count++;      // Meningkatkan count
        }
    }

    printf("Jumlah bilangan ganjil dari 1 sampai 100 adalah: %d\n", count);

    return 0;
}
```

Penjelasan Kode

a. Deklarasi dan Inisialisasi Variabel:

- o `int count = 0;` mendeklarasikan variabel `count` dan menginisialisasinya ke 0. Variabel ini digunakan untuk menghitung jumlah bilangan ganjil yang ditemukan.

b. Loop for:

- o `for (int i = 1; i <= 100; i++):` Loop dimulai dari 1 dan berlangsung hingga 100, meningkatkan nilai `i` setiap iterasi.
- o Ini adalah bentuk klasik dari loop `for` yang melibatkan tiga komponen:
 - **Inisialisasi:** `int i = 1` menetapkan nilai awal untuk iterator.
 - **Kondisi:** `i <= 100` menentukan batas atas loop.
 - **Peningkatan:** `i++` menambah nilai `i` sebesar satu setiap kali loop berlangsung.

c. Pernyataan Kondisional dalam Loop:

- o `if (i % 2 != 0):` Memeriksa jika `i` adalah bilangan ganjil menggunakan operator modulus `%`. Jika `i` dibagi 2 memiliki sisa, maka `i` adalah bilangan ganjil.
- o Jika kondisi ini terpenuhi, `count` akan diinkremen: `count++`.

d. Output:

- o `printf("Jumlah bilangan ganjil dari 1 sampai 100 adalah: %d\n", count);` mencetak jumlah total bilangan ganjil yang ditemukan dari 1 hingga 100.

Sub-Materi: Loop dengan `while` dan `do-while`

Program ini menggunakan loop `while` untuk meminta pengguna memasukkan password sampai mereka memasukkan password yang benar.

Contoh Program 1:

```
#include <stdio.h>
#include <string.h>

int main() {
    char password[20];
    int attempt = 0;
    const char correctPassword[] = "secret";

    printf("Enter the password: ");
    while (1) {
        scanf("%19s", password);
        if (strcmp(password, correctPassword) == 0) {
            printf("Password correct!\n");
            break; // Keluar dari loop
        } else {
            printf("Password incorrect, try again: ");
            attempt++;
            if (attempt >= 3) {
                printf("Too many failed attempts. Exiting.\n");
                break;
            }
        }
    }

    return 0;
}
```

Penjelasan Kode

a. Inisialisasi dan Deklarasi:

- `char password[20];` mendeklarasikan array karakter untuk menyimpan password yang diinput pengguna.
- `int attempt = 0;` variabel untuk melacak jumlah percobaan yang telah dilakukan.
- `const char correctPassword[] = "secret";` menyimpan password yang benar.

b. Loop while:

- `while (1):` Loop ini terus berjalan karena kondisinya selalu true sampai `break` dijalankan.
- `scanf("%19s", password);` mengambil input dari pengguna.
- `strcmp(password, correctPassword) == 0:` Memeriksa apakah password yang dimasukkan cocok dengan password yang benar.
- `break;` Menghentikan loop jika password benar.

c. Pengelolaan Kesalahan:

- Jika password salah, pengguna diminta untuk mencoba lagi.
- Jika pengguna salah memasukkan password lebih dari tiga kali, program menghentikan percobaan dan memberi tahu pengguna.

Contoh Program 2:

```
#include <stdio.h>

int main() {
    int number, sum = 0;

    do {
        printf("Enter a number (0 to stop): ");
        scanf("%d", &number);
        sum += number;
    } while (number != 0);

    printf("Total sum is: %d\n", sum);

    return 0;
}
```

Penjelasan Kode

1. Deklarasi dan Inisialisasi:

- `int number;` untuk menyimpan input pengguna.
- `int sum = 0;` untuk menyimpan jumlah kumulatif.

2. Loop do-while:

- Loop ini dijamin menjalankan blok kode di dalamnya setidaknya satu kali.
- `scanf("%d", &number);` mengambil bilangan dari pengguna.
- `sum += number;` menambahkan bilangan tersebut ke `sum`.
- Kondisi `while (number != 0);` mengecek apakah input adalah 0; jika ya, loop berhenti.

Bagian 3: Pointers dan Array

Sub-Materi: Konsep Dasar Pointer

Mengajarkan konsep dan penggunaan pointer dalam bahasa pemrograman C, yang memungkinkan pengaksesan langsung dan manipulasi nilai pada alamat memori tertentu. Program ini menggunakan pointer untuk menukar nilai antara dua variabel. Ini adalah contoh praktis yang mengilustrasikan bagaimana pointer dapat digunakan untuk memanipulasi data melalui referensi ke alamat memori.

Penggunaan pointer sangat penting dalam pemrograman C, terutama dalam kasus di mana kita perlu mengoptimalkan penggunaan memori dan performa, seperti dalam sistem tertanam atau aplikasi real-time. Pointer menyediakan cara yang sangat kuat dan fleksibel untuk berinteraksi dengan memori, memungkinkan programmer untuk mengontrol secara langsung bagaimana dan di mana data disimpan dan diakses.

Contoh Program:

```
#include <stdio.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int x = 10, y = 20;

    printf("Before swap: x = %d, y = %d\n", x, y);
    swap(&x, &y);
    printf("After swap: x = %d, y = %d\n", x, y);

    return 0;
}
```

Penjelasan Kode

a. Deklarasi Fungsi Swap:

- o **void swap(int *a, int *b):** Fungsi ini menerima dua pointer ke integer sebagai argumen. Pointer ini menunjuk ke alamat memori dari variabel x dan y.
- o **Dereferensi Pointer:** `int temp = *a;` mendereferensi pointer a untuk mendapatkan nilai dari variabel yang ditunjuk oleh a dan menyimpannya dalam variabel lokal temp.
- o **Menukar Nilai:** `*a = *b;` mendereferensi pointer a dan b untuk menukar nilai mereka. `*a` mendapatkan nilai dari `*b`, dan kemudian `*b` mendapatkan nilai dari temp, yang menyimpan nilai asli dari `*a`.

b. Main Function:

- o **Deklarasi Variabel:** `int x = 10, y = 20;` mendeklarasikan dan menginisialisasi dua variabel integer.

- **Panggil Fungsi Swap:** swap(&x, &y); memanggil fungsi swap dengan mengirimkan alamat dari x dan y. Operator & digunakan untuk mendapatkan alamat dari variabel, yang kemudian dijadikan pointer.
- **Cetak Hasil:** Menampilkan nilai x dan y sebelum dan sesudah pertukaran.

Sub-Materi: Pointer dalam Array

Mengajarkan konsep dasar tentang bagaimana array dapat dioperasikan dengan menggunakan pointer, serta penggunaan pointer untuk iterasi dan manipulasi elemen-elemen dalam array.

Program ini menggunakan array dan pointer untuk menghitung rata-rata nilai dari sejumlah elemen yang dimasukkan oleh pengguna. Ini merupakan contoh yang baik untuk memperlihatkan bagaimana pointer bisa digunakan untuk mengakses dan memanipulasi data dalam array.

Contoh Program:

```
#include <stdio.h>

int main() {
    int n, i;
    float sum = 0.0, average;
    printf("Masukkan jumlah elemen: ");
    scanf("%d", &n);
    int arr[n]; // Mendeklarasikan array dengan ukuran yang ditentukan

    printf("Masukkan %d elemen: \n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int *ptr = arr; // Pointer yang menunjuk ke array
    for (i = 0; i < n; i++) {
        sum += *(ptr + i); // Mengakses elemen array melalui pointer
    }

    average = sum / n;
    printf("Rata-rata nilai = %.2f\n", average);

    return 0;
}
```

Penjelasan Kode

a. Input Pengguna dan Deklarasi Array:

- Pengguna diminta untuk memasukkan jumlah elemen yang ingin dihitung rata-ratanya.
- Array `arr` diinisialisasi untuk menyimpan nilai-nilai tersebut, dengan ukuran yang ditentukan oleh pengguna melalui variabel `n`.

b. Mengisi Array:

- Loop `for` digunakan untuk mengumpulkan nilai dari pengguna dan menyimpannya dalam array `arr`.

c. Deklarasi dan Inisialisasi Pointer:

- `int *ptr = arr;` mendeklarasikan pointer `ptr` dan menginisiasikannya untuk menunjuk ke elemen pertama array `arr`.
- Pointer ini kemudian digunakan untuk mengakses semua elemen dalam array.

d. Perhitungan Menggunakan Pointer:

- Loop kedua `for` mengiterasi melalui array menggunakan pointer.
- `*(ptr + i)` adalah dereferensi pointer untuk mendapatkan nilai pada indeks `i` dari array. Ini secara efektif sama dengan `arr[i]`, tetapi menunjukkan bagaimana pointer dapat digunakan untuk navigasi array.

== Happy Coding :D ==